Data Structure

🧠 Topic 1: Data

✅ Definition:

Data is a collection of raw facts, figures, or values that represent information but have no meaning until they are processed.

> In computing, data can be in the form of numbers, characters, images, audio, or video.

🔄 Difference Between Data and Information:

Data	Information
Raw facts and figures	Processed and organized data
No context or meaning on its own	Meaningful and useful
Example: `45, 32, 67`	“The average score of the class is 48”

🔠 Types of Data:

1. **Numerical Data**
   - Example: `45, -9, 0.005`
   - Can be further divided into:
     - *Integer (int)*: `12, -100`
     - *Floating Point (float)*: `3.14, -0.89`
2. **Character Data**
   - Single symbols or characters
   - Example: `'a'`, `'B'`, `'9'`, `'#'`
3. **String Data**

- A sequence of characters
- Example: `"Hello"`, `"123abc"`

4. **Boolean Data**
    - Only two values: `true` or `false`
5. **Image, Audio, Video Data**
    - Complex data types, used in multimedia systems

🧩 Examples of Data in Daily Life:

Scenario	Example of Raw Data
Weather App	32°C, 1013 hPa, 72% humidity
Student Result	Marks in 5 subjects: `78, 90, 66, 81, 88`
E-Commerce Order	Product ID: 567, Quantity: 3, Price: ₹499
Attendance Register	Names, Present/Absent flags (1 or 0)
Bank Transaction	`Date: 10-07-2025`, `Amount: ₹2000`, `Type: Credit`

🗃️ How Data Is Represented in Memory:

In memory (RAM), everything is stored in **binary (0s and 1s)**.

For example:

- Character `'A'` → ASCII: `65` → Binary: `01000001`
- Integer `5` → Binary: `00000101`

💡 Why Understanding "Data" Is Important Before Learning Data Structures?

Because data structures are all about:

- **How to store data efficiently**
- **How to organize it**
- **How to retrieve or process it quickly**

So understanding the **types** and **nature** of data helps us choose the **right structure** to use.

🎓 Simple Exercise:

> 🔶 Identify the type of data in the following:
>
> 1. `true`
> 2. `"Delhi"`
> 3. `98.6`
> 4. `'Z'`
> 5. `2025`

🧠 Topic 2: Data Organization

✅ Definition:

Data organization refers to how data is arranged, structured, or grouped so that it can be used efficiently.

🧰 Why Is Data Organization Important?

Imagine you have 1000 student records. Without proper organization:

- It becomes hard to **search** for a specific student
- It's inefficient to **sort** them by marks
- It's difficult to **update** records quickly

> 💡 So, good organization makes data faster to access, easier to manage, and better for analysis.

🔄 Common Ways to Organize Data:

Organization Method	Description	Example
Linear	Data arranged in a sequence, one after another	Arrays, Linked Lists
Hierarchical	Data arranged in a tree-like structure	Folder systems, XML
Tabular	Rows and columns	Databases, Excel sheets
Graph-based	Data arranged using nodes and connections	Social networks, Maps
Hashed	Key-value pair for fast access	Hash Tables, Maps

🧩 Examples of Data Organization in Real Life:

Real-World Scenario	Type of Organization
Contacts in a phone	Alphabetical, Tabular
File storage on computer	Hierarchical (Folders/Subfolders)
Excel sheet of marks	Tabular (Rows = students, Columns = subjects)
Metro or Train Map	Graph (Stations = nodes, Routes = edges)
Dictionary lookup	Hashed (Key = Word, Value = Meaning)

Linear vs Non-linear Organization:

Aspect	Linear Data	Non-linear Data
Structure	Sequential	Branching or Hierarchical
Access Time	Generally slower (O(n))	Can be optimized (like O(log n))
Examples	Array, Linked List	Trees, Graphs

Memory Representation:

- **Contiguous** (like arrays): Easy to index but resizing is hard
- **Non-contiguous** (like linked lists): Flexible size but slower access

Key Takeaways:

- Organization affects **time complexity** (how fast operations run)
- Choosing the right data structure = Choosing the right way to **organize** your data

---

🎓 Exercise:

> Classify the following as Linear, Hierarchical, or Graph-based:
>
> 1. Your computer’s file explorer
> 2. A WhatsApp chat list
> 3. Delhi Metro route system
> 4. Student roll list in attendance register
> 5. Organization's employee chart

Topic 3: Data Access Methods

What Is Data Access?

Data Access refers to the way we retrieve, insert, or update data stored in a data structure or memory.

> Imagine you have books on a shelf. The way you pick a book — by number, name, or by checking one by one — is similar to different access methods in data structures.

Types of Data Access Methods:

There are **4 primary** ways to access data:

Access Method	Definition	Example
1. **Sequential Access**	Accessing elements **one-by-one** in order	Reading a book page by page
2. **Direct Access**	Jumping directly to a particular element using its **index**	Jumping to Page 45 of a book
3. **Random Access**	Accessing any memory location without following any sequence	RAM memory access
4. **Indexed Access**	Using an **index file/table** to find and access data faster	Book's index or search in Excel

1. Sequential Access

- You **start from the beginning** and go step by step.
- Common in: **Linked Lists**, Tapes, Files, Queues, Stacks
- **Slow** if the item is at the end

Example:

Searching for "Mango" in a grocery list:
[Apple, Banana, Grapes, Mango, Orange]
⇒ You check Apple → Banana → Grapes → Mango

2. Direct (Indexed) Access

- Each element has a unique position/index.
- Access using that index: like `arr[3]`
- Common in: **Arrays**

Example:

```
int arr[] = {10, 20, 30, 40, 50};
```

```
System.out.println(arr[2]);  // prints 30
```

- **Fastest** form of access (O(1) time)
- Needs memory to be stored **contiguously**

---

3. Random Access

- Similar to direct access, but more general.
- Used in hardware like **RAM** (Random Access Memory)
- Allows **any memory location** to be accessed instantly

Example:

- While using a laptop, if you open 3 different files at once, RAM accesses all of them quickly, not in sequence.

---

4. Indexed Access

- Uses an **index file** or **map** that holds the locations of actual data.
- Common in: **Databases**, Excel tables, Search engines

Example:

- Think of a dictionary:
  - You want to find the word “Zebra”.
  - You look at the index, it tells you “Page 210”
  - You directly jump there — this is **indexed access**.

Table Summary:

Method	Best Used For	Speed	Example Data Structure
Sequential Access	When processing all data in order	Slow	Linked List, Queue

Method	Best Used For	Speed	Example Data Structure
Direct Access	When location is known (by index)	Fast	Array
Random Access	For general fast memory access	Very Fast	RAM, Array
Indexed Access	For fast searching from huge data	Medium	Database, Excel

Easy Real-Life Analogy:

Real-Life Example	Access Method
Watching a movie on VHS tape	Sequential Access
Skipping to a scene in Netflix	Direct/Random Access
Searching for a word in dictionary	Indexed Access
Reading a magazine front-to-back	Sequential Access

Exercise:

> Match the access method:
>
> 1. Array → ?
> 2. Searching roll number in file → ?
> 3. Reading data from hard disk sectors → ?
> 4. RAM → ?
> 5. Playing audio from beginning to end → ?

Topic 4: Basics of Algorithm

What Is an Algorithm?

An **algorithm** is a **step-by-step** procedure or set of rules to solve a problem or perform a task.

> "Just like a recipe guides you in cooking a dish, an algorithm guides the computer to solve a problem."

Characteristics of a Good Algorithm:

Characteristic	Description
Input	It should take 0 or more inputs
Output	It should produce at least one output
Definiteness	Each step must be clear and unambiguous
Finiteness	It must end after a finite number of steps
Effectiveness	All steps must be basic enough to be done manually (theoretically)

📘 Real-Life Examples of Algorithms:

🍳 Example 1: Making Tea (Real Life)

Steps:

1. Boil water
2. Add tea leaves
3. Add sugar
4. Add milk
5. Stir and boil again
6. Strain the tea
7. Serve hot

👉 This is a **real-world algorithm** for making tea.

Find Maximum of 3 Numbers (Programming)

```
Input: a = 5, b = 9, c = 2
Step 1: Compare a and b → max = b
Step 2: Compare max and c → max = b
Output: 9
```

👉 This is a **simple algorithm** to find the maximum of 3 numbers.

How Do We Write Algorithms?

There are **two common methods**:

1. **Natural language (steps)**
2. **Pseudocode**

Example: Algorithm to check if a number is even or odd

Step Format:

```
Step 1: Take input number n
Step 2: If n % 2 == 0
          Print "Even"
        Else
          Print "Odd"
```

Why Do We Need Algorithms?

Because:

- Computers are fast but **need step-by-step instructions**
- Helps us solve **problems systematically**
- Leads to **efficient coding**

- Used in **real-world systems** like Google Search, ATM machines, Maps, etc.

Classification of Algorithms:

Type	Used When	Examples
Search Algorithms	To find data	Linear Search, Binary Search
Sort Algorithms	To arrange data	Bubble Sort, Merge Sort
Recursive Algorithms	When the problem can be divided into subproblems	Factorial, Tower of Hanoi
Greedy Algorithms	When local optimum leads to global optimum	Coin Change, Kruskal's Algo
Dynamic Programming	When overlapping subproblems exist	Fibonacci, 0/1 Knapsack
Divide and Conquer	When problem is broken down recursively	Merge Sort, Quick Sort

🧠 Fun Fact:

> Google Search, YouTube Recommendations, GPS, Online Shopping — all rely on clever algorithms behind the scenes!

🔁 Algorithm vs Program:

Algorithm	Program
Logic or steps to solve a problem	Implementation of algorithm in a language
Language-independent	Written in a specific language (like Java, C++)
Abstract	Concrete (code)

🧪 Quick Exercise:

> Write an algorithm (in steps) to:

> 1. Calculate the area of a rectangle
> 2. Find whether a given year is a leap year
> 3. Calculate the sum of first `n` natural numbers

Topic 5: Asymptotic Notations

What Are Asymptotic Notations?

Asymptotic notations are **mathematical tools** used to **analyze the performance of algorithms**, especially when input size grows very large.

> Instead of running the code, we study how many steps the algorithm takes.

Why Do We Need Asymptotic Notations?

- To understand how **efficient** an algorithm is as input grows
- To find out if your program will work well for **large inputs (n = 10^6 or more)**

Three Major Asymptotic Notations:

Notation	Represents	Tells us about
Big O (O)	Upper Bound (Worst Case)	Maximum time an algorithm will take
Omega (Ω)	Lower Bound (Best Case)	Minimum time an algorithm will take
Theta (Θ)	Tight Bound (Average or Exact case)	Average time or exact behavior

1. Big O Notation – Worst Case

- **Used most commonly**
- Describes the **maximum** number of steps algorithm takes as input increases
- Tells us how an algorithm behaves in the **worst-case scenario**

Example:

Linear Search:

- You search for an element in an array of size `n`
- **Worst case**: the element is at the end or not present
- Time: `n` comparisons → `O(n)`

🔶 *Binary Search:*

- Halves the input each time
- Time: `O(log n)`

Common Big-O Functions:

Time Complexity	Name	Example
O(1)	Constant Time	Accessing `arr[i]`
O(log n)	Logarithmic Time	Binary Search
O(n)	Linear Time	Linear Search
O(n log n)	Linearithmic Time	Merge Sort, Quick Sort (avg)
$O(n^2)$	Quadratic Time	Bubble Sort, Selection Sort
$O(2^n)$	Exponential Time	Recursive Fibonacci
O(n!)	Factorial Time	Traveling Salesman Problem

---

2. Omega Notation – Best Case

- Describes the **minimum time** an algorithm will take.
- Represents **ideal situation**, often theoretical.

Example:

Linear Search:

- If the target is the **first element**, it takes 1 step
- **Best case**: Ω(1)

Bubble Sort:

- If the array is **already sorted**, it can detect it in 1 pass
- Best case: Ω(n)

---

3. Theta Notation – Average Case / Tight Bound

- When an algorithm always takes the same time in best and worst case
- **Exact behavior** for many cases
- Used when time is tightly bound above and below

Example:

Linear Search:

If the target is in the **middle** → average $n/2$ steps

→ Time = Θ(n)

Binary Search:

Always takes around $\log_2 n$ comparisons

→ Time = Θ(log n)

📊 Visual Comparison:

```
perl
CopyEdit
Time Complexity Graph:
|
|         O(n²)
|          /
|         /
|        /
|       /
|      /
```

```
|      /      O(n)
|     /      /
|    /      /
|   /      /
|  /      /     O(log n)
| /      /     /
|/_____|_____/_________> Input Size (n)
  O(1)
```

💥 Misconception Alert:

- `O(n)` ≠ **exact** time → It's **at most** time
- A faster algorithm **in practice** may still have the same Big-O as a slower one (due to constant factors)

Easy Real-Life Analogy:

Scenario	Notation	Explanation
Delivery may take **up to 7 days**	O(7 days)	Worst case upper limit
Delivery may take **minimum 1 day**	Ω(1 day)	Best case lower limit
Delivery will take **exactly 3–5 days usually**	Θ(4 days)	Average expected time

---

🧪 Exercise:

> What is the Big O time complexity of:
>
> - a) Accessing `arr[i]`
> - b) A nested loop over `n`
> - c) Merge Sort
>
> 1. Write best case and worst case time for Linear Search.

Topic 6: Complexity of Algorithms

What Is Algorithmic Complexity?

Algorithmic complexity tells us **how much time** and **memory (space)** an algorithm will use, **based on the input size** `n`.

> "Efficiency is not just about doing the job right — it's about doing it fast and using less memory."

1. Time Complexity

Time Complexity is the amount of **time (number of steps or operations)** an algorithm takes to complete based on the size of input `n`.

> We count operations, not actual time.

How to Calculate Time Complexity?

We analyze how the **number of operations** grows with the input size.

Example 1: Linear Search

```
int arr[] = {2, 4, 6, 8, 10};
for (int i = 0; i < n; i++) {
    if (arr[i] == x)
        return i;
}
```

- Worst case: `x` is not found → `n` iterations
- Time Complexity: **O(n)**

Example 2: Nested Loop

```
for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
     System.out.print("*");
  }
}
```

- Outer loop: `n` times
- Inner loop: `n` times for each outer
- Time Complexity: **O(n²)**

Example 3: Binary Search

```
while(low <= high){
   mid = (low + high) / 2;
}
```

- Each step cuts input in half
- ✅ Time Complexity: **O(log n)**

📦 Common Time Complexities (Sorted by Speed):

Complexity	Name	Example Algorithm
O(1)	Constant	Accessing `arr[i]`
O(log n)	Logarithmic	Binary Search
O(n)	Linear	Linear Search
O(n log n)	Linearithmic	Merge Sort, Quick Sort

Complexity	Name	Example Algorithm
$O(n^2)$	Quadratic	Bubble Sort, Selection Sort
$O(2^n)$, $O(n!)$	Exponential / Factorial	Recursive Fibonacci, TSP

Important Notes:

- We ignore **constant factors**: $O(3n) \rightarrow O(n)$
- Focus only on **dominant terms**: $O(n + n^2) \rightarrow O(n^2)$

Pro Tip:

If input size increases and your program **slows down drastically**, it probably has **$O(n^2)$** or worse complexity.

2. Space Complexity

Space Complexity is the **amount of memory** (RAM) an algorithm uses to run — including:

- Input storage
- Output storage
- Temporary variables
- Recursion stack

📘 Example 1: Factorial (Recursive)

```
int fact(int n){
   if(n == 0) return 1;
   return n * fact(n - 1);
}
```

- Recursion depth = `n`

- Space Complexity: **O(n)** (due to call stack)

Example 1: Array Sum

```
int sum = 0;
for (int i = 0; i < n; i++) {
   sum += arr[i];
}
```

- Only one variable `sum`
- Space Complexity: **O(1)** (constant)

Time vs Space Trade-Off:

- Sometimes we **use more space** to **save time**

Quick Exercise:

> What is the time and space complexity of this code:

```
for(int i=0; i<n; i++){
    for(int j=0; j<n; j++){
        System.out.print(i + j);
    }
}
```

> What is the space complexity of:

```
int[] arr = new int[n];
```

🧠 Topic 7: Greedy Algorithm

✅ What Is a Greedy Algorithm?

A **Greedy Algorithm** builds up a solution **step-by-step**, always choosing the **best (most optimal) option at that moment**, with the hope that this leads to the **global optimum**.

> "Pick the locally best choice at each step, and hope it leads to the best overall solution."

🎯 Greedy Strategy in Simple Words:

Imagine you're at a shop with ₹100, and you want to buy as many items as possible. A **greedy strategy** would be:

🟢 Always buy the **cheapest item first**.

🔍 Key Idea:

- Solve the problem by making a **series of choices**.
- Each choice is the **best possible** at that moment.
- **Does not reconsider** previous decisions.

🧠 When Does Greedy Work?

Greedy works **only when** the problem has:

1. **Greedy-choice property**: A global solution can be arrived at by choosing local optimum.

2. **Optimal substructure**: A problem can be broken into smaller subproblems which can be solved optimally.

✅ Real-Life Examples of Greedy Strategy:

Scenario	Greedy Approach
Making change with coins	Always use the largest coin first
Scheduling maximum tasks	Pick tasks that finish earliest
Buying maximum items with limited money	Pick cheapest items first

---

📘 Example 1: Coin Change Problem

> Problem: You have coins of denominations {1, 2, 5, 10}, and you need to make change for ₹27 using the fewest coins possible.

Greedy Strategy:

- Pick highest possible coin each time:
  - ₹10 → Pick 2 → ₹20
  - ₹5 → Pick 1 → ₹25
  - ₹2 → Pick 1 → ₹27

    ✅ Answer: **3 coins**: [10, 5, 2]

🟠 **Note**: Greedy **may not work** for all coin systems (e.g., {1, 3, 4} for ₹6).

📘 Example 2: Activity Selection Problem

> You're given n activities with start and end times. Select the maximum number of non-overlapping activities.

Greedy Strategy:

- Sort by ending time
- Always pick the next activity that **starts after the previous ends**

✅ Time Complexity: **O(n log n)** (due to sorting)

🧠 Topic 8: Divide and Conquer

✅ What Is Divide and Conquer?

Divide and Conquer is a strategy where:

1. The problem is **divided** into smaller subproblems.
2. Each subproblem is **conquered** (solved recursively).
3. The solutions are then **combined** to form the final solution.

> "Break the problem into parts, solve each part independently, and combine the results."

🔍 Key Steps:

Step	What Happens
Divide	Break the problem into smaller subproblems
Conquer	Solve each subproblem (often recursively)
Combine	Merge the sub-solutions to get the final answer

🧠 When to Use It?

- Problem can be **broken into independent subproblems**
- Subproblems are **similar in nature**
- Useful when solving recursively and when combining helps build the final result

📘 Real-Life Analogy:

> You want to sort a stack of 100 cards:

- First, split it into 2 stacks of 50
- Sort each stack individually
- Finally, **merge** the two sorted stacks

That's **Divide and Conquer** in action!

📚 Famous Divide and Conquer Algorithms:

Algorithm	Time Complexity	Explanation
Merge Sort	O(n log n)	Divide → Sort → Merge
Quick Sort	O(n log n) avg, O(n^2) worst	Divide using pivot → Sort
Binary Search	O(log n)	Divide array → Search in one half

✅ Advantages of Divide and Conquer:

- Breaks large problems into manageable parts
- Naturally supports **recursion**
- Often **efficient** (e.g., `O(n log n)` sorting)

❌ Disadvantages:

- Overhead of **recursion** (stack space)
- Some problems cannot be split independently
- Requires **combining step** logic

Topic: Dynamic Programming (DP)

✅ Definition:

Dynamic Programming is a technique used in **algorithm design** to solve **problems with overlapping subproblems and optimal substructure** by **storing intermediate results** to avoid redundant work.

> 🔁 DP = Divide problem into subproblems → Store results → Reuse → Efficient solution.

🧩 Key Concepts:

1. Overlapping Subproblems

- The same subproblems are solved multiple times.
- Example: In Fibonacci, `fib(5)` calls `fib(4)` and `fib(3)` which again call `fib(2)`, etc.

2. Optimal Substructure

- The optimal solution of the problem depends on the optimal solutions of its subproblems.
- Example: Shortest path from A to C via B is optimal only if A→B and B→C are both shortest.

🔥 Why Use DP?

- Avoids **recomputation** → Speeds up algorithm
- Converts **exponential time algorithms** into **polynomial time algorithms**
- Solves **problems with constraints** efficiently

✅ Steps to Solve Any DP Problem:

1. **Identify if the problem has overlapping subproblems and optimal substructure**
2. **Define the state**: `dp[i]` = What does it represent?

3. **State transition**: Relation between `dp[i]` and smaller states
4. **Base case**: Smallest version of the problem
5. **Decide Memoization or Tabulation**
6. **Optimize space if needed**

✅ What is a Data Structure?

A **Data Structure** is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently.

> Think of it like different types of containers in your kitchen:
>
> - A **box** to store cookies (like an array)
> - A **basket** to put dirty clothes one on top of another (like a stack)
> - A **queue** at a billing counter where people come and go in order (like a queue)
> - A **family tree** showing parents, children, and grandchildren (like a tree)

🎯 Purpose of Data Structures

- Efficient **data storage**
- Faster **search, insertion, and deletion**
- Better **memory management**
- Helps in **algorithm design**

🧺 Types of Data Structures

Type	Example	Description
Linear	Array, Linked List, Stack, Queue	Data is arranged in a sequence
Non-Linear	Tree, Graph	Data is arranged in hierarchy or network
Hash-based	HashMap, HashSet	Stores data as key-value pairs
File-based	File, Database	Used for long-term storage

🌳 Data Structure Bifurcation

(Meaning: Categorizing/Dividing Data Structures)

🎯 Why Bifurcation is Important?

To solve problems efficiently, you must choose the **right type** of data structure. Bifurcation helps in understanding:

- **How data is stored**
- **How data is accessed**
- **What operations are easy or hard**

🧩 Bifurcation of Data Structures

1️⃣ Based on Structure Type:

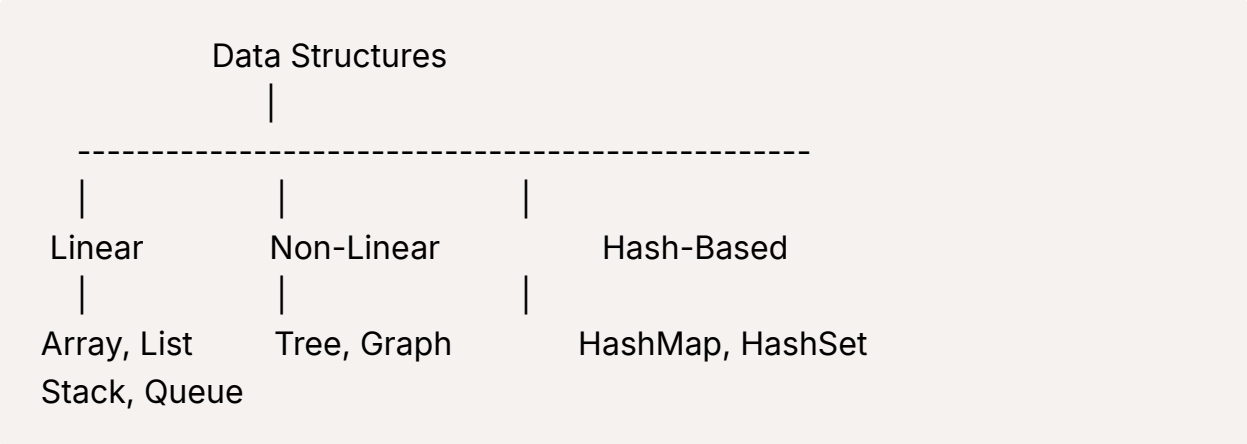
Category	Examples	Explanation
Linear	Array, LinkedList, Stack, Queue	Data stored **sequentially**, one after another
Non-Linear	Tree, Graph	Data stored in a **hierarchical** or **network** structure

2️⃣ Based on Storage Location:

Category	Examples	Explanation
Primitive DS	int, char, float	Basic data types
Non-Primitive DS	Array, List, Tree, etc.	Built using primitive DS

3️⃣ Based on Operation Access:

Category	Example	Operation Style
Static	Array	Fixed size, direct access
Dynamic	LinkedList	Size grows/shrinks, needs traversal



📌 General Applications of Data Structures

🔎 1. Arrays

- **Use Cases**:
  - Storing multiple values (e.g., marks of students)
  - Implementing matrices in scientific computation
  - Image processing (2D arrays)
- **Applications**:
  - Search & sort algorithms (Binary Search, Bubble Sort)

- Lookup tables

🔁 2. Linked Lists

- **Use Cases**:
  - Dynamic memory allocation
  - Implementation of queues and stacks
- **Applications**:
  - Music playlists (next/previous)
  - Image slideshow
  - Undo-redo functionality

📚 3. Stack

- **Use Cases**:
  - LIFO (Last In First Out) operations
- **Applications**:
  - Function call management in recursion
  - Parenthesis matching
  - Undo feature in editors
  - Expression evaluation (Infix → Postfix)

📮 4. Queue

- **Use Cases**:
  - FIFO (First In First Out) operations
- **Applications**:
  - CPU Scheduling (OS)
  - Print Queue
  - Handling requests in web servers

- BFS (Breadth-First Search)

🧠 5. Hash Table (HashMap, HashSet)

- **Use Cases**:
  - Fast access using keys
- **Applications**:
  - Implementing dictionaries
  - Caches
  - Symbol tables in compilers
  - Password verification systems

✅ STRING PROGRAMS

🔷 1. Find Length of String

```
java
CopyEdit
import java.util.Scanner;

public class StringLength {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String str = sc.nextLine();
        System.out.println("Length of the string is: " + str.length());
        sc.close();
    }
}
```

🔷 2. Copy a String

```
java
CopyEdit
import java.util.Scanner;

public class StringCopy {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String original = sc.nextLine();
        String copied = new String(original); // or simply copied = original;
        System.out.println("Copied string is: " + copied);
        sc.close();
    }
}
```

🔷 3. Compare Two Strings

```
java
CopyEdit
import java.util.Scanner;

public class StringCompare {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first string: ");
        String s1 = sc.nextLine();
        System.out.print("Enter second string: ");
        String s2 = sc.nextLine();

        if (s1.equals(s2)) {
            System.out.println("Strings are equal.");
        } else {
```

```
            System.out.println("Strings are not equal.");
        }

        sc.close();
    }
}
```

🔷 4. Concatenate Two Strings

```
java
CopyEdit
import java.util.Scanner;

public class StringConcat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first string: ");
        String s1 = sc.nextLine();
        System.out.print("Enter second string: ");
        String s2 = sc.nextLine();

        String result = s1.concat(s2); // or s1 + s2
        System.out.println("Concatenated string: " + result);
        sc.close();
    }
}
```

✅ ARRAY PROGRAMS

🔷 5. Search Element in Array

```
java
CopyEdit
import java.util.Scanner;

public class ArraySearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        System.out.print("Enter element to search: ");
        int target = sc.nextInt();

        boolean found = false;
        for (int i = 0; i < n; i++) {
            if (arr[i] == target) {
                System.out.println("Element found at index " + i);
                found = true;
                break;
            }
        }
        if (!found) System.out.println("Element not found.");
        sc.close();
    }
}
```

🔷 6. Sort Given Array

```
java
CopyEdit
import java.util.Arrays;
import java.util.Scanner;

public class ArraySort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        Arrays.sort(arr);
        System.out.println("Sorted array: " + Arrays.toString(arr));
        sc.close();
    }
}
```

🔷 7. Reverse Elements of Array

```
java
CopyEdit
import java.util.Scanner;

public class ArrayReverse {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
```

```
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        System.out.print("Reversed array: ");
        for (int i = n - 1; i >= 0; i--) {
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}
```

🔷 8. Addition of Elements of Array

```
java
CopyEdit
import java.util.Scanner;

public class ArraySum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        int sum = 0;
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
            sum += arr[i];
        }

        System.out.println("Sum of elements: " + sum);
        sc.close();
```

```
    }
}
```

🔷 9. Find Largest Element from Array

```
java
CopyEdit
import java.util.Scanner;

public class ArrayMax {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int max = arr[0];
        for (int i = 1; i < n; i++) {
            if (arr[i] > max) max = arr[i];
        }

        System.out.println("Largest element: " + max);
        sc.close();
    }
}
```

🔷 10. Find Smallest Element from Array

```
java
CopyEdit
import java.util.Scanner;

public class ArrayMin {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int min = arr[0];
        for (int i = 1; i < n; i++) {
            if (arr[i] < min) min = arr[i];
        }

        System.out.println("Smallest element: " + min);
        sc.close();
    }
}
```